KMIP on PKCS#11

From key server to key management service

Design · gap analysis · roadmap

41 / 53
KMIP 2.1 operations

816
tests, live against a token

82
features assessed vs market

23
gaps outside KMIP's reach

One protocol for every key store

BEFORE

N vendors, N integrations

- Every HSM and KMS exposed its own proprietary API
- Each database, backup tool and storage array needed bespoke work
- Changing vendor meant rewriting every integration

WITH KMIP

One protocol, many products

- A client speaks KMIP once and talks to any compliant server
- Oracle TDE, SQL Server, NetApp, VMware and Veeam all speak it
- Vendor substitution becomes a configuration change

What the standard actually is

- An OASIS standard: a wire protocol plus an object model, not a product
- TTLV — tag, type, length, value — a compact binary encoding carried over TLS
- Client/server, request/response, with batching; 53 operations defined in version 2.1
- Governs the whole life of a key: create, use, rotate, retire, destroy, and prove what happened

Managed objects, attributes, a lifecycle

OBJECT TYPES

What KMIP manages

- SymmetricKey
- PublicKey / PrivateKey
- Certificate
- SecretData
- OpaqueObject
- SplitKey

ATTRIBUTES

What describes them

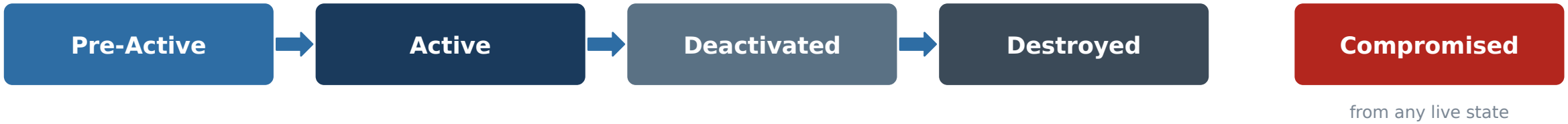
- Algorithm and length
- Cryptographic usage mask
- Name and custom x- attributes
- Links between keys
- Dates: activation, deactivation
- Owner and state

THE SUBTLE ONE

Cryptoperiod

KMIP defines no cryptoperiod attribute. The Deactivation Date is the end of the period — so enforcing a cryptoperiod means acting on a standard attribute, not inventing a parallel one.

Lifecycle



Deactivated keys may still decrypt and verify — retiring a key must not strand the data encrypted under it.

What the standard deliberately leaves out

KMIP DEFINES

Keys, and operations on keys

- Managed objects and their attributes
- The lifecycle state machine
- Cryptographic operations: encrypt, sign, MAC, derive, wrap
- Search over objects (Locate), batching, version negotiation

KMIP DOES NOT DEFINE

Everything around them

- Identities, roles, groups, grants — no user management at all
- Any HTTP or REST binding: TTLV over TCP only
- Tenancy, quotas, reporting, dashboards
- Administrative operations of any kind

The consequence

Every commercial KMS bolts an administrative plane onto its KMIP server. A KMIP layer is an interoperability surface — it is not, by itself, a product.

VERSIONS

2.1 → 3.0

2.1 (2019) is what this project implements. 3.0 CSD02 (May 2026) adds ML-KEM, ML-DSA, SLH-DSA and Encapsulate/Decapsulate.

THE DESIGN · 1 OF 3

KMIP 2.1 on a PKCS#11 token

ALONGSIDE

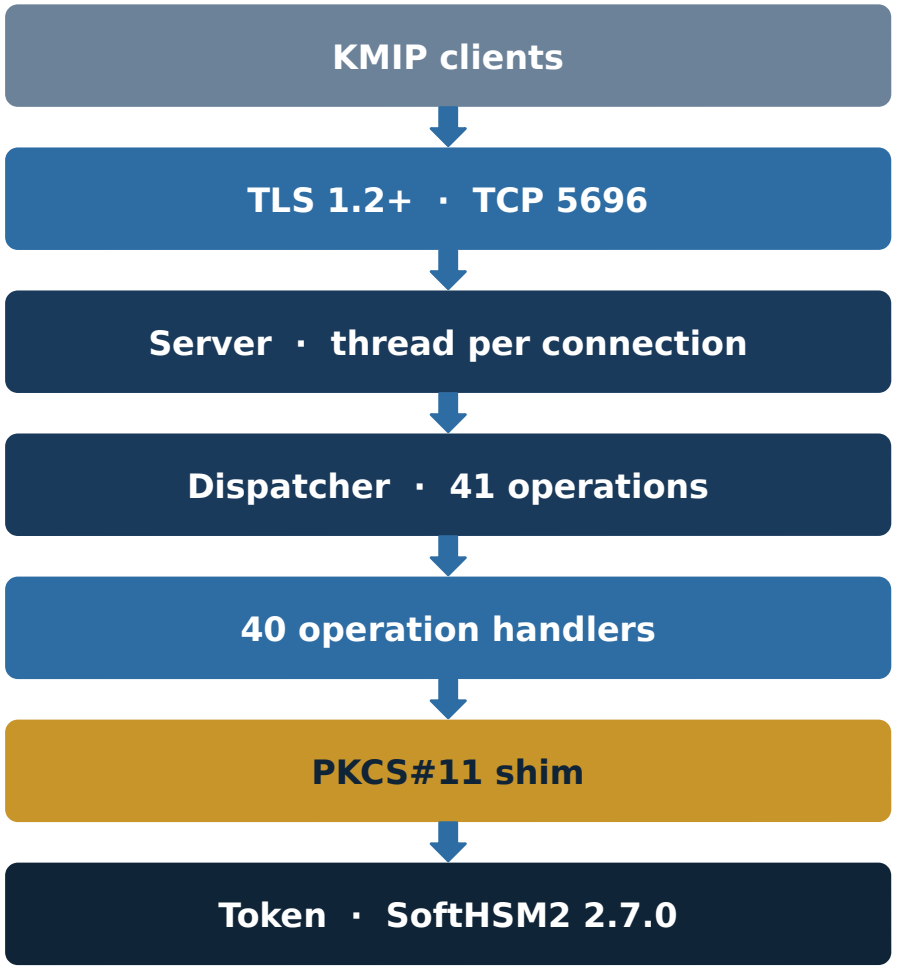
Metadata store

- SQLite, WAL mode
- KMIP attributes and state
- Identities, roles, grants
- Hash-chained audit log

SWAP POINT

Any PKCS#11 HSM

One file imports the binding. Pointing it at validated hardware needs no change above the shim — the capability probe reports whatever that token supports.



Oracle TDE · VMware · NetApp · scripts

enforced by default; mTLS optional

request caps, handshake timeout

authorization · dual control · audit · metrics

one file per KMIP operation

the only file that imports pkcs11

79 mechanisms · keys never leave

THE DESIGN · 2 OF 3

Two stores, one matched pair

IN THE TOKEN

Key material

- Generated inside the HSM, never leaves it
- Non-extractable; the usage mask is enforced by the token
- Referenced from the database only by CKA_ID

IN THE DATABASE

Everything KMIP needs that PKCS#11 has no room for

- Attributes, state, dates, names, links
- Owner, grants, groups, role allowlists, approvals
- The tamper-evident audit chain

THE EXCEPTION

Objects with no PKCS#11 representation

SecretData, OpaqueObject and SplitKey shares have no token object behind them, so their bytes would sit in the database in the clear. They are sealed with AES-256-GCM under a master key that is generated on, and never leaves, the HSM.

Back them up together

A database restored beside a different token is not a degraded backup. It is unreadable.

THE DESIGN · 3 OF 3

Where authorization, approval and audit live

The dispatcher wraps every one of the 41 operations in this pipeline.

THE HAZARD

Bypass

Anything that calls the store directly skips all four checks. Add a second transport carelessly and dual control silently stops applying to it — which is why a shared service layer must come before a REST API.

CONCURRENCY

One session per process

A PKCS#11 session pool was built, tested, and reproducibly segfaulted: python-pkcs11 calls C_Initialize(NULL), so the library's own thread safety is never enabled. Scale with pre-fork workers, never with threads. This is permanent design, not a stopgap.

CAPABILITY PROBE

Ask the token, don't assume

The shim reads the token's real mechanism list at startup and gates every dispatch on it. An unsupported algorithm fails cleanly and by name, instead of surfacing a raw PKCS#11 error from deep inside an operation.

A FULL KMS · 1 OF 2

Why a REST layer is not optional

01

The operations do not exist

Identities, roles, grants, approvals, cryptoperiods, audit search, backups — KMIP defines none of them.

02

A browser cannot speak TTLV

Binary over raw TCP. A browser has fetch and WebSocket. Something must translate — and that translator is the API.

03

The auth models differ

KMIP re-authenticates every request from a header credential. A console needs login, session, idle timeout, CSRF, revocation.

04

The read shapes are wrong

Locate returns bare identifiers with no paging, sort or count. A console table becomes N+1 round trips.

05

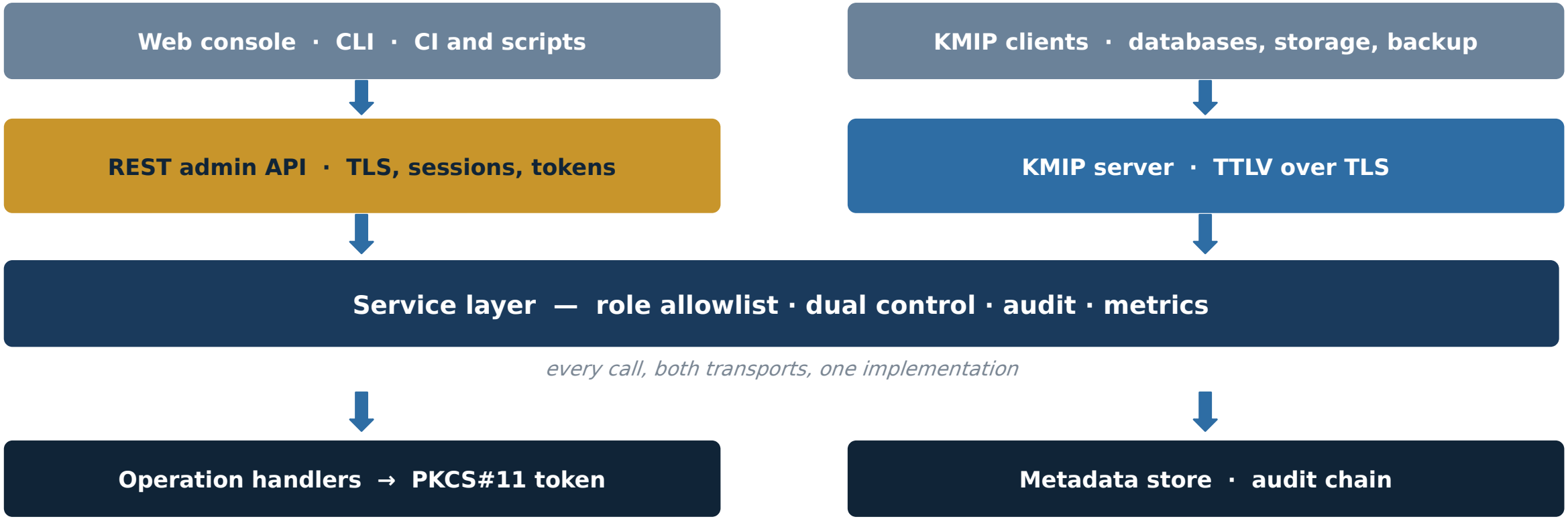
Not everything is object-centric

Approval queues, audit search, expiry reports, chain verification, health. None is an operation on a managed object.

The division of labour

KMIP stays the data plane — machines creating and using keys. REST becomes the control plane — people and CI administering the service. Duplicating crypto operations across both gives you two paths to Destroy, and two authorization implementations to keep in step.

One service layer, two transports



Endpoints: sessions and identity · keys and lifecycle · grants, groups, roles · approval queue · audit search · cryptoperiod and expiry reports · backups. No encrypt, decrypt or sign over REST in v1 — deliberately.

GAP ANALYSIS · 1 OF 3

82 features the market expects

32
covered

18
partial

32
not covered

23
cannot be closed in KMIP

Domain	Features	Covered	Partial	Gap
Key lifecycle and cryptography	15	**8**	4	**3**
Protocol and ecosystem integration	15	**1**	5	**9**
Identity and access control	13	**6**	1	**6**
Audit, compliance and assurance	11	**3**	2	**6**
Availability, scale and recovery	11	**3**	3	**5**
Operations and observability	9	**5**	1	**3**
Platform hardening	8	**6**	2	**0**
TOTAL	82	**32**	18	**32**

Requirements drawn from Thales CipherTrust, Entrust KeyControl, Fortanix DSM, Utimaco ESKM, Bloombase KeyCastle, Cosmian/Eviden, Securosys CyberVault, HashiCorp Vault and the cloud KMS services, plus NIST SP 800-57 and SP 800-152.

GAP ANALYSIS · 2 OF 3

What holds up

Cryptographic core

41 of 53 operations, capability-probed against the token; keys never leave the HSM.

Lifecycle and governance

State machine, cryptoperiod scheduler, scheduled rotation, dual control on Destroy and Export.

Access control

Per-identity scrypt credentials, admin role, ownership, per-object grants, groups, per-role allowlists.

Audit

Append-only two ways — triggers and a SHA-256 chain — with retention and a verifier that names the broken row.

Data at rest

AES-256-GCM envelopes under a non-extractable HSM master key, with rotation.

Backup and recovery

Online snapshot with token pairing, and a restore that proves a stored secret decrypts.

Several of these are done properly rather than nominally: the restore proves a secret decrypts rather than assuming it, and dual control is enforced in the store, so the requester cannot self-approve however the approval arrives.

GAP ANALYSIS · 3 OF 3

Where it falls short of a product

No REST API or console

Reporting, the console, self-service and encryption as a service are all clients of it.

No multi-tenancy

Names, searches, quotas and audit are global. Isolation means one deployment per tenant.

SQLite only

No clustering, replication or DR — the storage engine blocks them, not the protocol.

No rate limiting

The listener takes a backlog of 16 with no throttle. Flagged in the first review.

Policy lives in the server

Dual control is enforced here, not inside the token. Own the server and you own the policy.

Scaling stops at ~1.5×

Pre-fork workers help, but every audited operation serialises on one write lock.

No SIEM export or anchor

Auditors ask for both first. A consistent rewrite of the local log leaves no trace.

Starts unsealed by one PIN

The store needs the token, so a stolen database is inert. But one PIN holder is the ceremony.

Not FIPS or CC validated

Belongs to the token, not the code. Procurement, not engineering.

No post-quantum

Needs both a PQC token and KMIP 3.0. Neither has shipped in a release.

Red: absent and material. Amber: present with a named limit. Blue: outside this codebase's control.

ROADMAP · 1 OF 3

5 stages, in dependency order

A
Foundation
Steps 1–2 · closes 2

No new attack surface. Makes the rest safe to build.

B
The REST control plane
Steps 3–7 · closes 12

The API, then the console that uses it.

C
Enterprise fit
Steps 8–9 · closes 9

What an auditor and a security team ask for.

D
Scale, availability and tenancy
Steps 10–12 · closes 9

Needs infrastructure to verify.

E
Protocol and estate reach
Steps 13–14 · closes 2

Additive, independent of D, verifiable here.

How the stages are ordered

- By dependency, not by what is most visible
- Stage A ships no feature and unblocks everything after it
- Stage E is additive — it can run beside C or D
- Each step ships independently and is useful alone

THE RULE THAT MADE PHASES 0–5 WORK

Every step has a demonstrable gate

Not a checklist — a condition someone can watch you meet. A step is finished when its gate is demonstrated, not when its code is written.

ROADMAP · 2 OF 3

All 14 steps

1	Service layer and guard	—
2	Query and reporting layer	+2
3	HTTP transport, sessions and service accounts	+3
4	Read endpoints	—
5	Write endpoints, and the governance gaps	+4
6	Administration, OpenAPI, separation of duties	+3
7	Web console and self-service	+2
8	Enterprise identity and policy	+2
9	Audit externalisation, compliance and ceremony	+7
10	Storage abstraction and PostgreSQL	+1
11	High availability, failover and deployment	+6
12	Multi-tenancy	+2
13	KMIP JSON and XML encodings	+1
14	Certificate discovery and expiry monitoring	+1

A Foundation · B The REST control plane · C Enterprise fit · D Scale, availability and tenancy · E Protocol and estate reach +n is the gap-matrix features that step closes.

ROADMAP · 3 OF 3

Stages C to E — and what stays out

C — Enterprise fit

What an auditor and a security team ask for.

8. Enterprise identity and policy
9. Audit externalisation, compliance and ceremony

D — Scale, availability and tenancy

Needs infrastructure to verify.

10. Storage abstraction and PostgreSQL
11. High availability, failover and deployment
12. Multi-tenancy

E — Protocol and estate reach

Additive, independent of D, verifiable here.

13. KMIP JSON and XML encodings
14. Certificate discovery and expiry monitoring

Deliberately out of scope

- Secrets management
- Certificate lifecycle
- PKCS#11 provider for applications
- Tokenization / format-preserving encryption

Nothing ships that cannot be demonstrated. Where something could not be tested here, the plan says so rather than counting it as done.